

TECHNICAL TEST SUBMISSION
BADARUZZAMAN NAFIZ

Greenhouse IoT Backend

Reliable sensor ingestion and acknowledged device control
through one production-oriented backend.

FastAPI

PostgreSQL

Mosquitto

Docker Compose + Pytest



Three behaviors define success

The scope stays deliberately small and reviewable.

1

POST /sensor-data

Validate the payload, preserve recorded_at, and persist the reading in PostgreSQL.

2

POST /device-control

Accept device_id plus ON/OFF and publish a structured command through MQTT QoS 1.

3

GET /status

Probe the backend, SELECT 1 against PostgreSQL, and report the live MQTT connection state.

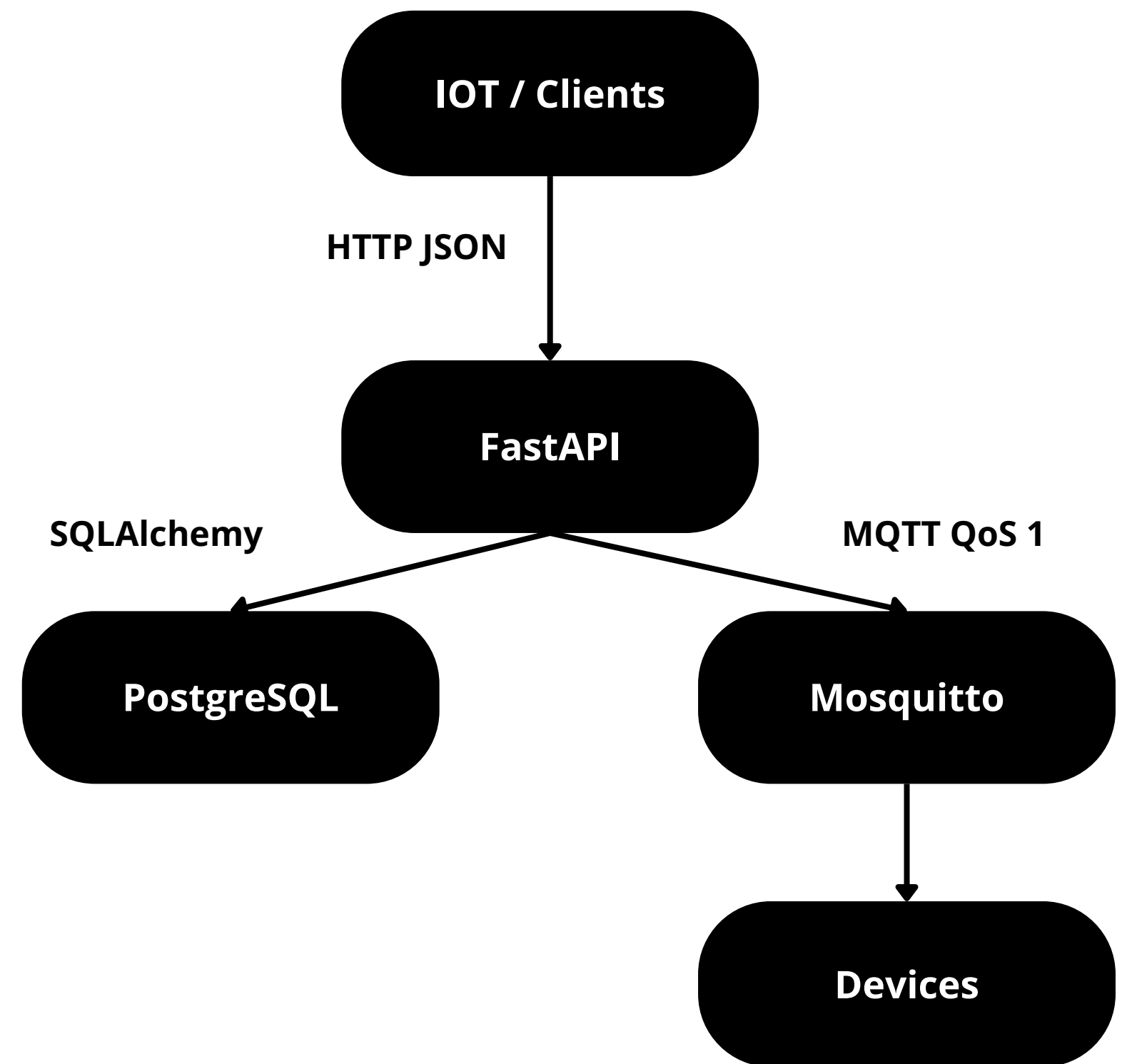
Deliverables

Source code · README · Design explanation · Presentation PDF

One backend, two transport responsibilities

Requirement interpretation

The implementation contract explicitly makes /sensor-data an HTTP endpoint, while MQTT is reserved for outbound device commands.



Sensors

POST `/sensor-data` Store Sensor Data

Devices

POST `/device-control` Control Device

System

GET `/status` Get Status

GET `/` Root

sensor_readings

id	device_id	temprature	humidity	recorded_at	created_at
PK	text	numeric	0-100	sensor time	backend time

The contract is explicit before implementation

1

POST `/sensor-data`

Validate and persist a reading

2

POST `/device-control`

Validate and publish a command

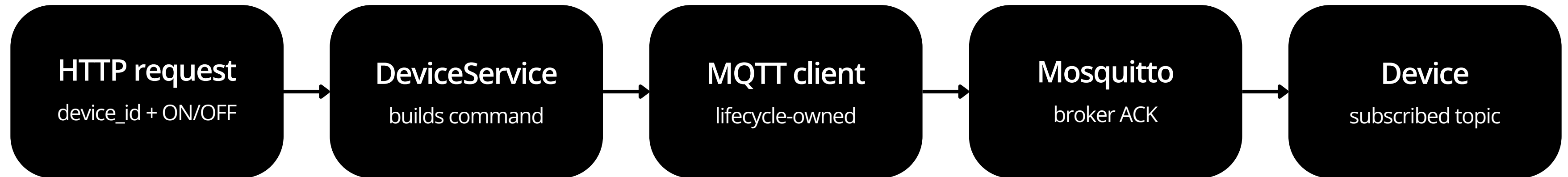
3

GET `/status`

Probe live dependencies

Persistent MQTT keeps commands reliable

The HTTP request returns only after publish acceptance is confirmed.



Topic

greenhouse/control/{device_id}

Payload

```
{ "device_id": "fan-001", "command": "ON",  
  "timestamp": "2026-09-19T10:35:00Z" }
```

Why QoS 1

Broker acknowledgement matters more for control commands than for best-effort telemetry. QoS 1 gives at-least-once delivery, so devices should treat commands idempotently.

Health is derived from real dependency checks

Failures are visible without exposing credentials or stack traces.

Healthy

Backend - Up
Database - Connected
MQTT - Connected

Degraded

Any failed DB query or disconnected MQTT client changes the aggregate status.

Consistent error envelope

```
{
  "success": false,
  "error": {
    "code": "MQTT_UNAVAILABLE",
    "message": "MQTT broker is unavailable"
  }
}
```

Malformed JSON, validation failures, database outages, and broker outages follow the same external structure. Internal detail stays in logs.

Testing proves the critical paths

Automated tests are paired with manual reviewer-style verification.

21 / 21 tests passing

Sensor validation · command validation · MQTT failure
· healthy/degraded status

MQTT subscriber evidence

```
$ mosquitto_sub -t greenhouse/control/# -v

greenhouse/control/fan-001
{"device_id":"fan-001","command":"ON",...}
```

Database persistence evidence				
id	device_id	temperature	humidity	stored through
2	reviewer-sensor	29.4	62.8	POST /sensor-data

Manual integration run retained separate recorded_at and created_at values.

Design choices keep the submission focused

Production expansion is possible without hiding today's failure modes.

Current decisions

FastAPI

Typed validation and automatic OpenAPI with a small code surface.

PostgreSQL + Alembic

Durable relational storage with explicit schema evolution.

Persistent MQTT + QoS 1

Lower request overhead and broker acknowledgement for commands.

Dependency-aware health

Partial failure is surfaced as degraded instead of masked.

Future improvements

Security

TLS, authentication, secret management, and per-device authorization.

Observability

Metrics, tracing, structured log aggregation, and alert thresholds.

Scale

Connection tuning, idempotency keys, and horizontal backend replicas.

Telemetry evolution

Add inbound MQTT consumption only when product requirements demand it.